

Kubernetes Networking in Our Lab

The four networks, why they must never overlap, how a packet really travels, what we fixed (ADR-003), CoreDNS, and how to design K8s networking in production

WHY THIS MATTERS

Networking is the hardest thing to change after a cluster exists. The Pod CIDR, Service CIDR, CNI and node addressing are effectively permanent; changing them usually means rebuilding the cluster. That's why interviewers love networking questions.

Contents

1. The building analogy
2. The four networks and why they must never overlap
3. The lab diagram with real addresses
4. Follow the packet: 6 journeys
5. What we changed in Phase 0 (ADR-003) and the split-horizon `k8s-api` name
6. CoreDNS: the cluster's phone directory
7. Designing K8s networking in production (on-prem, AWS, Azure)
8. Quiz and answer key

Lab: 3 VMs on VMware Workstation, 2 NICs each · Calico IPIP · written 2026-09-25

1. The building analogy

Think of your laptop as a **building** and the 3 VMs as **3 flats** inside it.

Real thing	Analogy
Windows laptop	The building owner's office
VMware NAT network (VMnet8), 192.168.64.0/24	The main road : connects the flats to the owner's office and, through the gate, to the internet
VMware host-only network, 10.0.0.0/24	A private corridor between the flats. The owner's office has no door onto it
Pod network 172.17.0.0/16	The rooms inside each flat . Each flat numbers its rooms from its own block
Service network 10.96.0.0/16	Reception phone extensions : numbers that don't belong to any room; dial one and reception forwards you to a real room
CoreDNS	The phone directory : ask for "discount-service", get the extension number

2. The four networks

#	Network	Range	Real or virtual?	Who hands out addresses	Used for
1	LAN / NAT (ens33)	192.168.64.0/24	Real (VMware NIC)	VMware DHCP (.128-.254)	Internet, Windows → cluster , MetalLB IPs later
2	Cluster network (ens34)	10.0.0.0/24	Real (VMware NIC)	Static (.100 / .1 / .2)	API server, kubelet, etcd, Calico tunnels
3	Pod network	172.17.0.0/16	Virtual (Calico)	Calico, one /26 block per node	Every pod's IP
4	Service network	10.96.0.0/16	Purely virtual : on no interface	API server	Stable ClusterIPs in front of pods

2.1 Why they must never overlap

Linux decides where to send a packet by looking up the destination in its **routing table**. If two networks share addresses, the lookup can pick the wrong one.

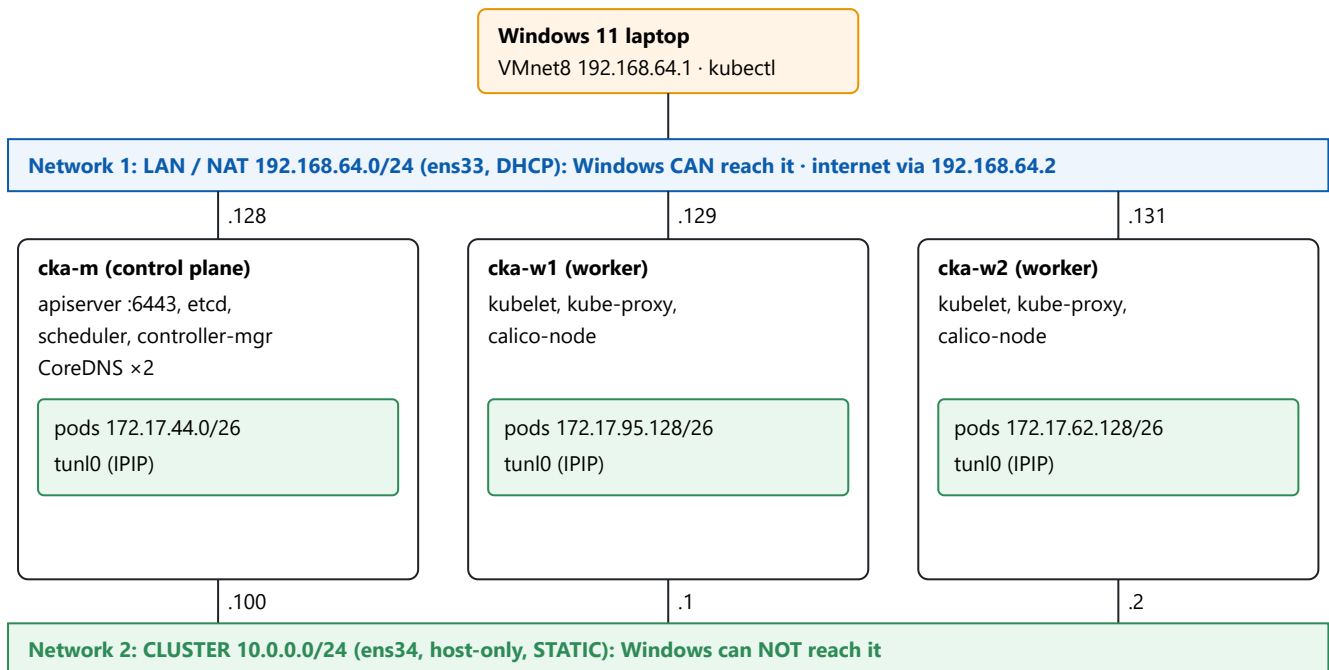
Before our fix: ens34 = 10.0.0.x/8 → "10.0.0.0 to 10.255.255.255 is on ens34"
Service range 10.96.0.0/16 is INSIDE that → overlap
After: ens34 = 10.0.0.x/24 → only 10.0.0.0 - 10.0.0.255 → no overlap

It happened to work before, because kube-proxy rewrites Service IPs before packets leave, but any hiccup would have sent packets down the wrong interface. That's the worst kind of bug to debug.

TWO MORE OVERLAPS TO REMEMBER

- **172.17.0.0/16** is our Pod CIDR *and* Docker's default bridge. So **never install Docker on the nodes**. We build images on Windows (Docker Desktop) instead.
- MetalLB IPs must be **below 192.168.64.128**, because VMware's DHCP hands out .128-.254.

3. The lab diagram



Network 3: PODS 172.17.0.0/16, virtual, carried inside network 2 by IPIP tunnels

Network 4: SERVICES 10.96.0.0/16, virtual, only iptables rules · 10.96.0.1 = kubernetes API · 10.96.0.10 = kube-dns

4. Follow the packet: 6 journeys

① kubectl on Windows → API server

Windows —(network 1)—→ 192.168.64.128:6443 on cka-m → kube-apiserver
kubeconfig uses the name k8s-api, which Windows resolves to 192.168.64.128 (hosts file)

② Worker → API server (kubelet reporting "I'm alive")

cka-w1 10.0.0.1 —(network 2)—→ k8s-api = 10.0.0.100:6443 static on both ends: DHCP can't break it

③ Pod on w1 → pod on w2 (IPIP: an envelope inside an envelope)

Outer envelope (node → node, network 2): from 10.0.0.1 → to 10.0.0.2

Inner envelope (pod → pod): from 172.17.95.130 → to 172.17.62.131
the real data (e.g. order-service calling discount-service)

How does cka-w1 know that 172.17.62.128/26 lives on cka-w2? Calico's BGP agent (**BIRD**) shares routes between nodes:

172.17.62.128/26 via 10.0.0.2 dev tunl0 proto bird
172.17.95.128/26 via 10.0.0.1 dev tunl0 proto bird

④ Pod → Service IP (e.g. DNS at 10.96.0.10)

```
pod → dst 10.96.0.10:53
      iptables rules (written by kube-proxy on EVERY node) rewrite the destination:
      10.96.0.10 → one of the CoreDNS pods      ← this is DNAT
      → then travels like journey ③
```

No machine owns 10.96.0.10. It's a rule, not an interface. So you **can't ping a Service IP** (ICMP isn't covered by the rules), but you **can** connect to its port.

⑤ Pod → internet

```
pod → node rewrites the source to its own IP (SNAT / masquerade) → ens33 → VMware NAT →
internet
```

⑥ Browser → GaramChai (Phase 7, not built yet)

```
browser → https://garamchai.test (hosts file → a MetalLB IP such as 192.168.64.50)
        → MetalLB answers ARP "that IP is at cka-w1's MAC" → Traefik → app pods
```

LAB GOTCHA: PING DOESN'T WORK INSIDE PODS

CRI-O's default capabilities exclude `NET_RAW`, which `ping` needs, so ping fails with "permission denied" (containerd's defaults include it). Test with TCP instead: `wget`, `nc`, `curl`.

5. What we changed in Phase 0 (ADR-003)

Change	Before	After	Why
ens34 mask	/8	/24	Removed the overlap with the Service range
kubelet --node-ip	not set → kubelet picked ens33 (DHCP)	10.0.0.100 / .1 / .2	The node's InternalIP is now static. The API server uses it for <code>kubectl logs/exec</code> (port 10250)
Calico node IP	auto-detected	<code>IP_AUTODETECTION_METHOD=cidr=10.0.0.0/24</code>	With 2 NICs, every component must pick the same NIC
API endpoint	a node IP	<code>controlPlaneEndpoint: k8s-api:6443</code>	A stable name; HA possible later

5.1 Split-horizon name resolution for `k8s-api`

The same name resolves to a **different address depending on who asks**:

Who asks	File	<code>k8s-api</code> resolves to	Why
The nodes	<code>/etc/hosts</code>	10.0.0.100	Cluster traffic stays on the static network
Windows	<code>C:\Windows\System32\drivers\etc\hosts</code>	192.168.64.128	The only network Windows can reach

Because `kubeadm` puts `k8s-api` into the API server's certificate, the TLS check passes from both sides. No certificate hacks, no `insecure-skip-tls-verify`.

INTERVIEW LINE

"In my lab the nodes had two NICs, and each Kubernetes component was picking a different one. I pinned everything (API server advertise address, kubelet node-ip, Calico autodetection) to a static /24, which also removed an overlap between a /8 node network and the Service CIDR. The API endpoint is a DNS name with split-horizon resolution, so the cluster doesn't depend on DHCP."

6. CoreDNS: the cluster's phone directory

Pods get a new IP every time they restart, so apps never use pod IPs. They use **names**, and CoreDNS turns a name into the Service's stable IP.

```
kubectl -n kube-system get deployment coredns # 2 replicas
kubectl -n kube-system get svc kube-dns # 10.96.0.10: still called kube-dns, a historical name
kubectl -n kube-system get configmap coredns -o yaml # its config, the "Corefile"
```

The kubelet writes every pod's `/etc/resolv.conf`:

```
nameserver 10.96.0.10
search garamchai.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

You ask for	You get	GaramChai example
<svc> (same namespace)	Service ClusterIP (via the search list)	http://discount-service
<svc>.<namespace>	Service ClusterIP	postgres.garamchai-data
<svc>. <ns>.svc.cluster.local	Service ClusterIP (full name, FQDN)	our test: kubernetes.default.svc.cluster.local → 10.96.0.1
Headless Service	All pod IPs directly	Postgres StatefulSet (Phase 5)
<pod>.<svc>. <ns>.svc.cluster.local	One specific StatefulSet pod	postgres-0.postgres.garamchai-data...
google.com	Forwarded to the node's upstream DNS	calling an external API

TWO DNS GOTCHAS WE HIT OR SHOULD KNOW

- **busybox nslookup:** versions after 1.28 ignore the `search` line, so `nslookup kubernetes.default` returns NXDOMAIN even when DNS is fine. Use the FQDN, or busybox:1.28.
- **ndots:5:** names with fewer than 5 dots try the search suffixes *first*, so `google.com` causes 3–4 wasted lookups. Fixes: trailing dot (`google.com.`), `dnsConfig` per pod, or NodeLocal DNSCache.

6.1 "A pod can't resolve names": the checklist

1. Are the CoreDNS pods running? `kubectl -n kube-system get pods -l k8s-app=kube-dns`
2. Does the `kube-dns` Service have endpoints? (`get endpointslices`)
3. From a test pod, look up the **FQDN**.
4. Check `/etc/resolv.conf` inside the failing pod.
5. Read the CoreDNS logs.
6. Is a NetworkPolicy blocking UDP/TCP 53? (We'll meet this in Phase 8.)

7. Designing K8s networking in production

7.1 The 7 questions, in order

#	Question	Typical mistake
1	IP plan: node, Pod, Service ranges that don't collide with each other, the corporate LAN, VPNs, peered VPCs or other clusters	Using 10.0.0.0/8 or 172.17.0.0/16 without checking; ranges too small
2	CNI: overlay (Calico, Cilium) or routable pod IPs (AWS VPC CNI, Azure CNI)? NetworkPolicy support?	Flannel (no NetworkPolicy); running out of subnet IPs
3	HA topology: nodes across zones; 3 control-plane nodes; a load-balanced API endpoint	A single master; API address tied to one machine
4	Ingress: L4 load balancer + L7 controller, TLS, WAF	NodePorts exposed to users
5	Egress: NAT gateway, fixed egress IPs for partners	Random egress IPs a bank can't whitelist
6	DNS: CoreDNS, corporate DNS forwarding, ExternalDNS	Corporate names not resolvable from pods
7	Security: private subnets, firewalls/security groups, default-deny NetworkPolicy, private API endpoint	Flat network; API server on the internet

Also check **MTU** (overlays add headers: IPIP 20 bytes, VXLAN 50; a mismatch gives "small requests work, big ones hang") and **firewall ports:** 6443 API, 2379–2380 etcd, 10250 kubelet, 30000–32767 NodePort, 179/TCP BGP, IP protocol 4 for IPIP, 4789/UDP VXLAN.

7.2 On-prem vs AWS vs Azure

Topic	On-prem (like our lab)	AWS (EKS)	Azure (AKS)
Pod networking	Calico/Cilium overlay, or BGP to the switches	AWS VPC CNI: pods get real VPC IPs (size subnets, prefix delegation, or secondary CIDR)	Azure CNI Overlay (common default), Azure CNI with VNet IPs, or Cilium
API HA	You build it: 3 masters + VIP (kube-vip / keepalived + HAProxy)	Managed by AWS	Managed; private-cluster option
type: LoadBalancer	MetalLB or hardware (F5)	AWS Load Balancer Controller (NLB / ALB)	Azure Load Balancer; App Gateway for L7
Egress	Corporate firewall / proxy	NAT Gateway per AZ; VPC endpoints	NAT Gateway or Azure Firewall
Biggest trap	Nobody plans IPs; DHCP node IPs	Running out of pod IPs in small subnets	Picking a mode that eats VNet IPs

60-SECOND INTERVIEW ANSWER

"I start with the IP plan, because it's the one thing you can't change later: node, pod and service ranges that don't overlap with each other or anything the cluster connects to. Then the CNI: an overlay like Calico or Cilium on-prem, or the cloud CNI on EKS/AKS with subnets sized for pod IPs. Then HA: nodes across zones and a load-balanced API endpoint. Then traffic in (load balancer + Ingress or Gateway with TLS) and out (NAT with fixed egress IPs). Then DNS, then security: private subnets, security groups, default-deny NetworkPolicies, a private API endpoint. In my lab I found and fixed a /8 node network overlapping the Service CIDR and node IPs on DHCP."

8. Quiz

1. Name our four networks with their ranges, and say which ones are purely virtual.
2. Why did `ens34` at `/8` cause a problem, even though everything "worked"?
3. Why must we never install Docker on the nodes?
4. Why must MetalLB IPs be below `192.168.64.128`?
5. Explain IP-in-IP encapsulation using the envelope picture. Which addresses are on the outer and inner envelopes?
6. How does `cka-w1` know which node owns `172.17.62.128/26`?
7. Why can't you ping a ClusterIP, even though you can connect to its port?
8. What does kube-proxy actually do when a pod connects to `10.96.0.10`?
9. What is split-horizon DNS, and how does `k8s-api` use it in our lab?
10. With 2 NICs per node, which three settings did we pin to the cluster network?
11. `ping` fails inside a pod with "permission denied". Is the network broken? How do you test instead?
12. What is the Service `kube-dns`, and why is it called that when we run CoreDNS?
13. `nslookup kubernetes.default` returns NXDOMAIN, but the FQDN works. What happened?
14. What does `ndots:5` do, and why can it hurt at scale?
15. A pod can't resolve names. List your troubleshooting steps in order.
16. On AWS EKS with the VPC CNI, what's the biggest networking trap?
17. Which network ranges must not overlap when you design a production cluster?
18. Symptom: small HTTP requests work, large ones hang. What's the likely cause?

Answer key

1. LAN/NAT 192.168.64.0/24 (ens33), cluster 10.0.0.0/24 (ens34), Pods 172.17.0.0/16, Services 10.96.0.0/16. Pods are virtual (Calico overlay); Services are purely virtual (only iptables rules).
2. /8 covered 10.0.0.0–10.255.255.255, which includes the Service range 10.96.0.0/16. Overlapping ranges can route packets down the wrong interface. It only worked because kube-proxy rewrites Service IPs first.
3. Docker's default bridge uses 172.17.0.0/16, the same as our Pod CIDR, which would break pod routing.
4. VMware's DHCP hands out .128–.254; a MetalLB IP inside that pool could be given to another machine too.
5. The pod's packet (inner: 172.17.95.130 → 172.17.62.131) is wrapped in a node packet (outer: 10.0.0.1 → 10.0.0.2), sent over the cluster network, and unwrapped on the other node.
6. Calico's BGP agent (BIRD) shares each node's /26 block, which appears as a route: `172.17.62.128/26 via 10.0.0.2 dev tunl0 proto bird`.
7. No interface owns a ClusterIP; only iptables/IPVS rules for its TCP/UDP ports exist. ICMP isn't translated.
8. Nothing at connection time. It has already written iptables rules on every node; the kernel DNATs 10.96.0.10 to one of the CoreDNS pod IPs.
9. The same name resolves differently depending on who asks. Nodes: `k8s-api` → 10.0.0.100 (hosts file); Windows: → 192.168.64.128. Both match the certificate, because `k8s-api` is in it.
10. API server `advertiseAddress` (10.0.0.100), kubelet `--node-ip`, and Calico `IP_AUTODETECTION_METHOD=cidr=10.0.0.0/24`.
11. Not necessarily. CRI-O drops the `NET_RAW` capability that ping needs. Test with TCP (`wget`, `nc`, `curl`).
12. The Service in front of the CoreDNS pods (10.96.0.10). The name is historical, from before CoreDNS replaced kube-dns, and kept for compatibility.
13. `busybox nslookup` (after 1.28) ignores the search domains, so it looked up the literal name. DNS itself was fine.
14. Names with fewer than 5 dots try the search suffixes first, so external names cause several wasted lookups. That multiplies DNS load. Fix with a trailing dot, `dnsConfig`, or `NodeLocal DNSCache`.
15. CoreDNS pods running → kube-dns endpoints exist → FQDN lookup from a test pod → the pod's `resolv.conf` → CoreDNS logs → NetworkPolicy blocking port 53.
16. Running out of IP addresses, because every pod takes a real VPC IP. Size subnets properly, use prefix delegation or a secondary CIDR.
17. Node/VPC subnets, Pod CIDR, Service CIDR, and every network the cluster will connect to: corporate LAN, VPNs, peered VPCs, other clusters, on-prem data centres.
18. An MTU mismatch: the overlay header makes large packets too big, and they get dropped.